L22: Scenario Comparison for Inventory Policies

Simulation Without a Black Box

Outline

Learning Objectives

What Is a Scenario?

30 Replications and CIs

Paired Comparison with CRN

Bar Chart with CI Error Bars

Selecting Under a Service-Level Constraint

compare_scenarios Usage

Key Takeaways

Learning Objectives

By the end of this lecture you will be able to:

- Define a scenario as a factor×level combination and enumerate a design.
- Run 30 replications per policy and compute 95% confidence intervals for cost.
- Apply Common Random Numbers (CRN) to reduce variance in policy comparisons.
- Identify the best policy subject to a fill-rate constraint.
- Use `simdes.analysis.scenarios.compare_scenarios` for structured output.

What Is a Scenario? Factor × Level

Definition

A **scenario** (also: design point, configuration) is a complete assignment of values to all controllable factors. Each scenario produces one or more replications.

Example: (s,S) policy comparison

Scenario	s	S
A	10	80
B	20	100
C	30	100
D	20	120
E	30	120

Why not just vary one factor?

- s and S interact: high s + high S is different from high s + low S.
- Full factorial: explore all combinations.
- Here: 2 levels of s × 2 levels of S + a low-budget baseline = 5 scenarios.

30 Replications per Policy: Confidence Intervals

```python
import numpy as np
from simdes.models.inventory import SsInventory
from simdes.analysis.scenarios import compare_scenarios

scenarios = [
    {"s": 10, "S": 80},
    {"s": 20, "S": 100},
    {"s": 30, "S": 100},
    {"s": 20, "S": 120},
    {"s": 30, "S": 120},
]

results = compare_scenarios(
    model_cls=SsInventory,
    scenarios=scenarios,
    n_reps=30,
    metric="total_cost",
    base_seed=42,
    use_crn=True,
)
```

95% CI for mean cost

$\bar{X} \pm t_{29,0.025} \cdot S/\sqrt{30}$ where $t_{29,0.025} \approx 2.045$

Paired Comparison Using Common Random Numbers

Common Random Numbers (CRN)

Drive all scenarios with the *same* demand stream (same seed per replication). The difference $D_j = X_{Aj} - X_{Bj}$ has reduced variance because demand is identical.

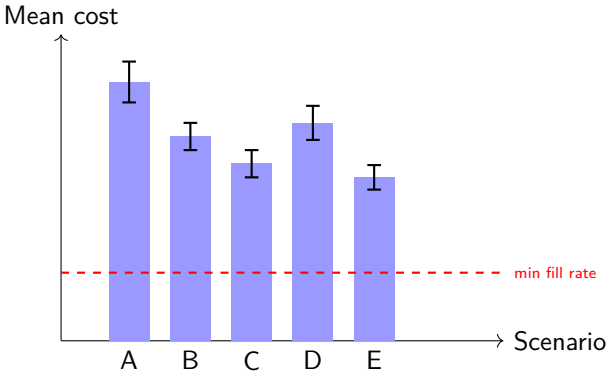
Implementation with `rng.spawn`:

1. Create n_{reps} base seeds.
2. For each scenario, run replication j with seed j — common to all policies.
3. Compute $\bar{D} \pm t \cdot S_D/\sqrt{n}$ for pairwise tests.

When CRN can backfire

CRN induces *positive* correlation between scenarios — beneficial for variance reduction. But if the mapping of random numbers to model events differs across scenarios (different number of orders), correlation can be negative. Verify: $\text{Var}[\bar{D}] < \text{Var}[\bar{X}_A - \bar{X}_B]$.

Bar Chart: Mean Cost with 95% CI Error Bars



Scenario E (s=30, S=120) has the lowest mean cost and CIs do not overlap with A or B — statistically significant. Read the chart jointly with fill-rate constraint.

Selecting the Best Policy Under a Fill-Rate Constraint

Constraint: fill rate $\geq 97\%$.

Scenario	Mean cost	Fill rate	Feasible?
A	395	91%	No
B	348	96%	No
C	318	99%	**Yes**
D	362	97%	**Yes**
E	335	99.5%	**Yes**

Winner: Scenario E — lowest cost among feasible policies.

Constrained selection procedure

1. Discard all scenarios with estimated fill rate $< 97\%$ (even if mean is low).
2. Among feasible scenarios, pick the one with the lowest mean cost.
3. Report the 95% CI for cost *and* the CI for fill rate.
4. If CIs overlap, flag as "statistically indistinguishable" and choose the safer option.

compare_scenarios: Full API Usage

```python
from simdes.analysis.scenarios import compare_scenarios

results = compare_scenarios(
    model_cls=SsInventory,
    scenarios=[{"s": s, "S": S}
               for s in [10,20,30]
               for S in [80,100,120]],
    n_reps=30,
    metrics=["total_cost", "fill_rate", "avg_onhand"],
    base_seed=42,
    use_crn=True,
    alpha=0.05,          # CI level
    constraint={"fill_rate": (">=", 0.97)},
)

print(results.best_scenario)      # Scenario with min cost s.t. constraint
print(results.summary_table)      # pandas DataFrame, one row per scenario
results.plot_bar("total_cost")    # matplotlib bar chart with CI
```

Key Takeaways

Core ideas from L22

1. A **scenario** is a complete factor assignment; enumerate all combinations before running.
2. Use **30 replications** per scenario and report 95% CIs — never report a single-run result.
3. **CRN** (same demand seed per replication) typically reduces CI width by 30–70% in inventory comparisons.
4. Apply the **constraint first**: discard infeasible policies before selecting on cost.
5. `simdes.analysis.scenarios.compare_scenarios` automates replication, CI, and constraint filtering.

This completes Module M06. Next module: output analysis, warmup detection, and variance reduction (M07).